

Edge cases (a request that fails validation) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proving it properly), **Advanced** (the sneaky way a partial write can still happen). Read Basic first even if this looks trivial — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This exercise doesn't ask you to write new route code — it asks you to prove something about the `sqlite_persistence` route you already built: that a bad request never reaches your database at all.

Things to use:

- `curl -X POST localhost:8000/chat -H "Content-Type: application/json" -d '{}'` — sends a request missing the required `message` field.
- Check the status code in the response — should be `422`.
- `sqlite3 runs.db "SELECT COUNT(*) FROM runs;"` (or a small Python script) to check the row count before and after.

Intermediate Version

The reason this works without you writing anything new: FastAPI parses and validates the request body against `ChatRequest` *before* your route function's body ever runs — so if `message` is missing, your function's `INSERT` statement is never reached. That's the entire mechanism keeping bad input away from your database, and it's automatic, not something you coded yourself.

The exact pieces:

- `requests.post(url, json={})` (from the `requests` library) is often easier to script than `curl` for checking a status code programmatically: `response = requests.post("http://localhost:8000/chat", json={}); assert response.status_code == 422`.
- `response.json()` on a 422 reply shows FastAPI's structured error detail — a list of exactly which field(s) failed and why, e.g. `{"detail": [{"loc": ["body", "message"], "msg": "Field required", ...}]}`.
- Count rows both before and after the bad request, and assert they're equal — don't just eyeball it once.

Advanced Version

The interesting question isn't "does FastAPI's automatic 422 happen" — it does, reliably, for a genuinely malformed body. The interesting question is: **could a request that passes validation still**

cause a partial write, if your route's logic fails partway through, after some but not all of the work is done?

Think about a route that does two separate database operations per request (e.g., insert a "run started" row, then later update it to "completed") — if the second operation fails, you can be left with a row that's forever stuck in "started," which is a silent partial write that 422 handling alone doesn't prevent, because the request was valid all along.

This is genuinely worth testing on purpose, on top of the required-field 422 case:

- Send a request with `message` present but with the wrong *type* (`{"message": 123}` instead of a string) — confirm this also 422s, since Pydantic checks types, not just presence.
- If your route does more than one database write per request, use a `try/except` around all of them together, or wrap them in a single SQLite transaction, so a failure partway through rolls back everything instead of leaving a half-done row.
- `conn.execute("BEGIN") ... conn.commit()` or `conn.rollback()` on failure — the transaction boundary that makes "all or nothing" actually true.

Difference between Basic, Intermediate, and Advanced: Basic proves the required case by hand with `curl` and a row count. Intermediate explains *why* it works (validation runs before your function body) and scripts the check properly with `requests` and an assertion instead of eyeballing it. Advanced goes past the case FastAPI handles for free, and asks what happens to a request that passes validation but fails partway through multi-step logic — the case where a silent partial write can still happen, and where a transaction, not automatic validation, is the real fix.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
count rows in runs table -> before_count

send POST /chat with {} (no message field)

check: status code is 422
count rows in runs table again -> after_count

check: before_count == after_count
```

Here's almost the whole thing, as a small script against your already-running server:

```
# sqlite_persistence_practice.py – Edge cases section
import sqlite3
import requests

conn = sqlite3.connect("runs.db")
before = conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

response = requests.post("http://localhost:8000/chat", json={})
print(response.status_code)

after = conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]
print(before, after)
```

Expected output:

```
422
0 0
```

(or whatever your row count already was before — the point is `before == after`.)

Intermediate Version

```

open a connection just to check row counts (separate from the app's own connection)

before_count = SELECT COUNT(*) FROM runs

response = POST /chat with {} # missing "message"
assert response.status_code == 422

after_count = SELECT COUNT(*) FROM runs
assert before_count == after_count

print the structured error FastAPI sent back, to see what a 422 actually looks like

```

```

# sqlite_persistence_practice.py – Edge cases section
import sqlite3
import requests

conn = sqlite3.connect("runs.db")

def row_count():
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

before = row_count()

response = requests.post("http://localhost:8000/chat", json={})
assert response.status_code == 422, f"expected 422, got {response.status_code}"

after = row_count()
assert before == after, f"row count changed: {before} -> {after}"

print("422 confirmed, no row written")
print(response.json())

```

Expected output:

```

422 confirmed, no row written
{'detail': [{'type': 'missing', 'loc': ['body', 'message'], 'msg': 'Field required'}]}

```

Advanced Version

```

run the same before/after check as Intermediate, plus:

send a second bad request: {"message": 123} # wrong type, not missing
assert this also 422s

```

```

if your /chat route does more than one database write per request:
    wrap those writes in a single transaction
    simulate a failure between the writes on purpose
    confirm nothing partial got committed

```

Here's most of it — the transaction test needs your own route's shape filled in:

```

# sqlite_persistence_practice.py – Edge cases section
import sqlite3
import requests

conn = sqlite3.connect("runs.db")

def row_count():
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

before = row_count()

# Case 1: missing field
response = requests.post("http://localhost:8000/chat", json={})
assert response.status_code == 422
assert row_count() == before

# Case 2: wrong type, not missing
response = requests.post("http://localhost:8000/chat", json={"message": 123})
assert response.status_code == 422
assert row_count() == before

print("both bad-input cases confirmed: 422, no row written")

# your turn: if /chat does 2+ writes per request, write a version of run_chat()
# that raises partway through a multi-step operation, and confirm your route's
# transaction handling leaves zero partial rows behind, not a half-written one

```

Difference between Basic, Intermediate, and Advanced: Basic proves the one required case by hand. Intermediate turns that into a real, assertion-based check and looks at what FastAPI's 422 body actually contains. Advanced adds a second bad-input shape (wrong type, not just missing) and asks the harder question 422 handling alone can't answer: what happens to a request that's perfectly *valid* but fails partway through multi-step logic — where a transaction, not FastAPI's automatic validation, is what actually prevents a silent partial write.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)